Leveraging Open-Source LLMs for Zero-Shot Vulnerability Detection: A Comparative Analysis

Nicola Capuano, Vincenzo Carletti(✉), Pasquale Foggia, Giuseppe Parrella, and Mario Vento

Department of Computer Engineering, Electrical Engineering and Applied Mathematics, University of Salerno, Via Giovanni Paolo II, 132, 84084 Fisciano, SA, Italy
{ncapuano,vcarletti,pfoggia,gparrella,mvento}@unisa.it

Abstract. The rapid expansion of the Internet of Things (IoT) has brought significant security challenges, primarily due to vulnerabilities in the firmware of IoT and network devices, which is predominantly written in low-level programming languages such as C and C++. Traditional vulnerability detection techniques, including static and dynamic analysis, rely on manual rules and face limitations with modern software complexity. Machine learning and deep learning offer promising alternatives by identifying vulnerability patterns from data but require extensive labeled datasets and may struggle to generalize to new code patterns, particularly in the diverse and evolving IoT landscape. This work evaluates open-source Large Language Models (LLMs) for zero-shot vulnerability detection in C and C++ using prompt engineering, benchmarking their performance against traditional static analysis tools and deep learning methods. Leveraging a newly introduced real-world test set, our experimental results confirm the inherent complexity of the vulnerability detection task and highlight the importance of further research aimed at enhancing the effectiveness of existing approaches in realistic contexts.

1 Introduction

The rapid proliferation of the Internet of Things (IoT) has interconnected billions of devices across sectors such as healthcare, transportation, and manufacturing [20]. While this integration fosters innovation and convenience, it also introduces significant security challenges for both devices and network infrastructures. The firmware of IoT and network devices, often written in low-level programming languages like C and C++, enables efficient hardware interactions but is particularly prone to security vulnerabilities. Common issues include buffer overflows, improper memory management, and race conditions [12,26,27]. These *vulnerabilities* can facilitate large-scale attacks, such as the formation of botnets [3,5], and pose significant risks to physical safety in critical applications. Exploiting

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025
L. Barolli (Ed.): AINA 2025, LNDECT 249, pp. 13–25, 2025.
https://doi.org/10.1007/978-3-031-87775-9_2

these weaknesses can lead to data breaches and disruptions of essential services, highlighting the need for effective mechanisms to detect and mitigate software vulnerabilities. Traditional vulnerability detection approaches, including *static* and *dynamic analysis*, have been employed to identify security flaws in software. Traditional Static Application Security Testing (SAST) tools [4,29,31,32] examine source or binary code without execution to identify potentially vulnerable patterns, while Dynamic Application Security Testing (DAST) tools execute programs in controlled environments to detect malicious behaviors. However, these methods rely heavily on manually crafted rules developed by domain experts and require meticulous calibration to reduce false positives and undetected vulnerabilities [15,34]. As software systems become more complex, traditional rule-based methods struggle to keep pace with emerging security threats. In response to the challenges of vulnerability detection, the research community has increasingly adopted machine learning, particularly Deep Learning (DL) techniques, to minimize dependence on domain experts. Natural Language Processing (NLP)-based methods treat code as text, utilizing tokenization and embedding with neural architectures like Recurrent Neural Networks (RNNs) and Transformers [9]. Meanwhile, Graph Neural Network (GNN)-based approaches represent code as graphs using structures such as Abstract Syntax Trees (ASTs) and Control Flow Graphs (CFGs), capturing essential structural information for vulnerability detection [1,7,10,17,19,39]. Large Language Models (LLMs), pre-trained on vast datasets of natural language and code, have emerged as powerful tools for software vulnerability detection [36]. Their broad training enables effective generalization across various domains, applicable to downstream tasks through *fine-tuning* or *prompt engineering*. In this work, we investigate the potential of open-source LLMs for vulnerability detection in C and C++ source code. Specifically, we employ *prompt engineering* techniques in a *zero-shot* setting, avoiding the need for additional training phases. A *zero-shot* approach refers to leveraging a model to perform a task without providing it with task-specific examples or additional training, relying solely on the general knowledge it has already acquired during pretraining. This strategy is aligned with scenarios where security professionals may lack the resources or expertise to fine-tune a model. Instead, they can efficiently leverage an existing model by utilizing well-designed prompts. We compare the performance of open-source Traditional SAST tools, Deep Learning models, and LLMs. The deep learning models are trained and validated on the *MIVIA Vulnerable Functions Source Code* (MVFSC) [6] dataset, while the LLMs operate in a zero-shot setting. The MVFSC dataset consolidates samples from various publicly available datasets, including critical software projects like the Linux Kernel and FFmpeg, ensuring its relevance to IoT applications. The study introduces a novel dataset comprising real-world software vulnerabilities excluded from the training and validation sets of deep learning models. This dataset enables a comprehensive evaluation of the models' generalization abilities by testing them on realistic and previously unseen scenarios. Experiments assess the performance of Traditional Static Application Security Testing (SAST) tools, Deep Learning models, and Large Language Models (LLMs) in

detecting vulnerabilities within this dataset. Results reveal the task's complexity and the challenges these methods face in practical settings. The analysis highlights the limitations of current approaches and stresses the need for further research to improve vulnerability detection, especially in critical areas like IoT security.

Contribution. In summary, the main contributions of this work are:

- A systematic comparison between Traditional SAST tools, Deep Learning models, and open-source LLMs for vulnerability detection;
- A rigorous zero-shot evaluation of LLMs using prompt engineering, offering insight into how these models perform without task-specific fine-tuning;
- The introduction of a novel dataset containing real-world software vulnerabilities, providing an extensive benchmark for future research on robust vulnerability detection in IoT and similar critical domains.

Organization. This paper is structured as follows. Section 2 reviews the state-of-the-art approaches and representations developed for source code analysis and vulnerability detection. Section 3 describes the experimental setup, including the dataset and the methods utilized in the study. The results and their analysis are presented in Sect. 4. Finally, Sect. 5 draws some concluding remarks and outlines potential directions for future research.

2 Related Work

A *vulnerability* represents a specific instance of a *weakness* in software. Weaknesses refer to flaws in design or implementation that can potentially allow a security threat. To standardize terminology in the security domain, weaknesses are cataloged in the *Common Weakness Enumeration (CWE)*, which provides unique identifiers and detailed descriptions for known weaknesses. Publicly disclosed vulnerabilities, on the other hand, are indexed in the *Common Vulnerabilities and Exposures (CVE)* program, with data made available through platforms like the *National Vulnerability Database (NVD)* [27]. Vulnerability detection methods can be broadly categorized into two types: *(i)* binary classifiers, that only determine whether the code is vulnerable or not, and *(ii)* multi-label classifiers, that also identify the kind of weakness (represented by its CWE identifier). In this section, we review recent advancements in this area, focusing on two main approaches: NLP-based methods and Graph-Based methods.

Natural Language Processing Approaches. NLP-based methods treat source code as textual data, represented as sequences of *tokens*, which are then embedded into a high-dimensional vector space using techniques such as Word2Vec [25]. These token embeddings are processed by neural architectures, traditionally dominated by Recurrent Neural Networks (RNNs), to capture sequential dependencies within the code. On top of these embeddings, classifiers like Multi-Layer Perceptrons (MLPs) are employed to identify vulnerabilities and, where applicable, predict corresponding CWE identifiers. The introduction

of *Transformers* [9] has significantly advanced NLP-based approaches for code analysis. Transformers, unlike RNNs, utilize an attention mechanism [30] that enables them to focus on the most relevant parts of the input sequence. For instance, μVulDeePecker [40] enhances Long Short-Term Memory (LSTM) networks with a *code attention* mechanism to detect vulnerable regions, such as API calls and control statements. This method also introduces the concept of *gadgets*, which are semantically related code fragments capturing data and control dependencies. More recent advances have leveraged Transformer architectures explicitly designed for source code. VulBERTa [16], based on the RoBERTa Transformer [21], is tailored for analyzing C and C++ source code. The training of VulBERTa is performed in two stages: a pre-training phase to learn general-purpose code representations from unlabeled datasets and a fine-tuning phase where the model adapts these representations for vulnerability detection. In the fine-tuning stage, VulBERTa employs either an MLP or a Text Convolutional Neural Network (TextCNN), resulting in two variants: VulBERTa-MLP and VulBERTa-TextCNN.

LLMs have recently emerged as powerful tools for software vulnerability detection, leveraging their extensive training on diverse datasets to understand complex code structures. They can be adapted to specific tasks either through prompt engineering or fine-tuning. Fine-tuning involves updating the model's parameters with domain-specific data to enhance precision in tasks like vulnerability detection, assessment, and localization. Alternatively, few-shot learning employs natural language prompts or minimal examples to enable quick adaptability without extensive retraining, though it may not achieve the same depth as fine-tuning. In the case of source code, most available open-source LLMs are trained on tasks such as code generation, documentation generation, or code completion, rather than specifically on vulnerability analysis. For Python codebases, SecureQwen [24] exemplifies the effectiveness of fine-tuning, employing a decoder-only Transformer architecture with extended context length to target diverse coding patterns. For C/C++ codebases, the GRACE framework [23] highlights the benefits of prompt engineering, combining semantic and syntactic code features with techniques such as in-context learning and chain-of-thought (COT) prompting. Similarly, the DLAP framework [35] integrates pre-trained deep learning models with LLM prompting to improve detection accuracy, utilizing in-context learning prompts derived from pre-trained features and enhancing reasoning through LLM-generated COT steps. Despite advancements, significant challenges persist, as the datasets used to train LLMs for software vulnerability detection may exhibit poor quality, noisy labels, and duplication [11]. These issues severely limit the models' ability to generalize to unseen projects and real-world scenarios, even with fine-tuning, often resulting in performance akin to random guessing under rigorous evaluation settings [11].

Graph-Based Approaches. While NLP approaches treat source code as text, graph-based methods leverage the inherent structural information present in code. Representing code as graphs allows capturing both semantic and syntactic relationships. Common graph representations include Abstract Syntax

Trees (ASTs) [14], Control Flow Graphs (CFGs) [2], and Program Dependency Graphs (PDGs) [13]. However, these individual representations often lack sufficient expressiveness for complex code structures. To address this, *Code Property Graphs (CPGs)* [34] combine ASTs, CFGs, and PDGs into a unified representation, allowing simultaneous analysis at multiple abstraction levels. Graph Neural Networks (GNNs) [33] enable direct processing of graph-structured data by generating embeddings for nodes, edges, or entire graphs. These embeddings encode both structural and semantic information, enabling downstream tasks like vulnerability detection. For example, Devign [39] employs Gated Graph Neural Networks (GGNNs) to process CPGs, generating node embeddings that are subsequently passed through convolutional layers for classification. Similarly, ReVeal [7] combines word embeddings from Word2Vec with GNNs to process CPGs, producing graph-level embeddings for vulnerability prediction. VulDetectionGNN [37] operates by transforming source code into a graph representation, where tokens serve as nodes and co-occurrence relationships within a fixed window define the edges. Node features are initialized using embeddings from pre-trained models, such as CodeBERT or GraphCodeBERT, ensuring semantically rich representations. A hybrid Graph Neural Network (GNN) architecture, combining Graph Convolutional Networks (GCNs) for local context and Gated GNN (GGNNs) for global dependencies, is employed to extract features. Residual connections stabilize the learning process, while a self-attention layer focuses on critical code patterns. The resulting graph embeddings are processed through pooling layers and a fully connected output layer to perform vulnerability classification with high accuracy and generalization capability.

3 Experimental Framework

The main objective of this work is to assess whether traditional SAST tools, deep learning approaches and LLMs in zero-shot settings can be considered effective tools for source code vulnerability analysis in a practical settings. In this section, we describe the preparation of the dataset and how it has been used to train and compare the selected approaches.

3.1 Dataset Preparation

The *MIVIA Vulnerable Functions Source Code (MVFSC)* [6] dataset is a comprehensive resource specifically designed for source code vulnerability analysis. It integrates and processes samples from four publicly available datasets: Devign [39], Big-Vul [12], DiverseVul [8], and ReVeal [7]. The dataset contains 396,130 training samples and 19,762 testing samples, including 15,656 vulnerable functions annotated with specific CWE categories, supporting both binary and multi-class classification tasks. Its realistic class imbalance mirrors real-world distributions, making it suitable for training robust and practical models. In this work, the MVFSC dataset was employed for training and validating deep learning-based models. Specifically, the provided training and test sets of

MVFSC were used as the training and validation datasets, respectively, ensuring a consistent baseline for evaluating model performance.

Table 1. Comparison of the MVFSC dataset used for training and validation with the newly introduced MVFSC Test Set. Each sample in the datasets corresponds to a single C/C++ function.

Dataset	Vulnerable Functions	Non-Vulnerable Functions
MVFSC Training Set [6]	31,985	364,145
MVFSC Validation Set [6]	1,916	17,786
MVFSC Test Set	993	972

To further assess the generalization capabilities of the proposed methods, we introduced a new test set, referred to as the *MVFSC Test Set*. This decision was motivated by observations in the literature, which highlighted significant issues in existing datasets, such as noisy labels, poor data quality, and high duplication rates, that can compromise the evaluation of vulnerability detection models [11]. This test set was created by carefully curating samples from diverse and widely used open-source C/C++ projects. Vulnerabilities were identified through reliable sources such as CVE details and the NVD [27], and vulnerable samples were collected from platforms like GitHub and GitLab. Wherever possible, the corresponding fixed versions of the code were also retrieved, ensuring the inclusion of both vulnerable and non-vulnerable instances. This process focused on major CWE categories to enhance the relevance of the test set. To ensure high data quality, meticulous preprocessing steps were applied. Redundant samples were removed through hashing-based deduplication, while irrelevant elements such as excessive spaces and comments were eliminated without altering the syntax or semantics of the code. Each sample was annotated with comprehensive metadata, including its CWE identifier, project name, and a binary vulnerability label. The final MVFSC Test Set consists of 1,965 samples with an almost equal distribution of vulnerable and non-vulnerable functions (see Table 1). By focusing on real-world vulnerabilities and their fixed counterparts, this test set serves as a robust benchmark for evaluating the generalization capabilities of vulnerability detection methods, complementing the training and validation datasets from MVFSC.

3.2 Evaluated Approaches

In this section, we explain the different methods valuated in this work. We include evaluation of traditional SAST tools, deep learning methods, and open source LLMs.

Static Analysis Tools. In this work, traditional SAST tools were employed as baselines to evaluate the performance of more advanced vulnerability detection approaches. Specifically, we included Semgrep [29] and Flawfinder [31] in

our analysis. For Semgrep, a diverse range of community-defined rule sets was utilized, encompassing categories such as OWASP Top Ten, CWE Top 25, and security audit standards. This configuration ensured broad coverage across various vulnerability types, enhancing the tool's applicability for comprehensive evaluation. In the case of Flawfinder, multiple operational modes were explored to optimize its detection capabilities, with particular emphasis on sensitivity adjustments tailored to specific CWE categories. The final assessment was based on the configuration that achieved the best balance between precision and recall, evaluated using the MVFSC validation dataset.

Deep Learning Approaches. In this work, we considered three deep learning methods for vulnerability detection: VulDetectionGNN [37], and VulBERTa [16] in its two variants, VulBERTa-TextCNN and VulBERTa-MLP. All models were fine-tuned for binary classification (determining whether code is vulnerable or not) using the MVFSC training set. For both VulBERTa models, we fine-tuned using a learning rate of 1×10^{-5}, training for up to 50 epochs with a batch size of 64. The AdamW optimizer [22] was employed with a weight decay of 0.01, and a cosine learning rate scheduler was applied with a warmup phase constituting 10% of the total training steps. The VulDetectionGNN [37] model was fine-tuned using a learning rate of 5×10^{-5} and trained for up to 100 epochs with a batch size of 64. We used the AdamW optimizer without weight decay and a linear learning rate scheduler with a warmup phase of 10% of the total training steps. To address class imbalance in the dataset, we implemented oversampling of the minority class during training for all models, ensuring balanced representation in each batch. For all neural networks, the output of the fully connected layer was passed through a softmax function, and a binary cross-entropy loss function was used for training.

Open-Source Large Language Models. In this work, we investigated the use of open-source LLMs for zero-shot vulnerability detection through prompt engineering. Our approach leverages LLMs pre-trained on source code-related tasks, such as code generation, without requiring additional fine-tuning. Specifically, we evaluated three models: CodeGeex4 [38] with 9 billion parameters, Qwen-Coder 2.5 [18] with 14 billion parameters, and CodeLlama [28] with 34 billion parameters.

To enable these models to identify vulnerabilities, we developed a carefully designed prompt that framed the task with concrete examples. The prompt included labeled code snippets demonstrating both vulnerable and non-vulnerable code, as illustrated in Fig. 1. Following the examples, the models were tasked with analyzing an input code snippet to classify its vulnerability status using a binary label (`0` for not vulnerable or `1` for definitely vulnerable). To ensure consistency and conciseness, the response format was strictly constrained to avoid any extraneous information.

```
Here are sampled examples from the training data:
Example 1:
    code example 1
    This code is not vulnerable (label: 0).
Example 2:
    code example 2
    This code is not vulnerable (label: 0).
Example 3:
    code example 3
    This code is vulnerable (label: 1).
Example 4:
    code example 4
    This code is vulnerable (label: 1).

Now, given the following code:
    code-to-analyze

Is this code definitely vulnerable?
Respond with "1" for definitely vulnerable or
"0" for not vulnerable or potentially vulnerable.
If definitely vulnerable, provide the CWE number.
Do not include any additional information.
```

Fig. 1. Example of a prompt used for vulnerability detection. Code snippets provided as examples to the LLMs will replace the blue placeholders, while the code of the function to be tested will replace the red placeholder.

4 Experimental Results

In this section, we present the results of the experimental analysis, followed by a discussion of the findings. Before delving into the interpretation of the outcomes, we introduce the metrics used to evaluate performance and clarify the definitions of false positives and false negatives, which are critical in understanding the reported results. The metrics considered in the analysis include well-known measures such as *accuracy* (Eq. 1), *precision* (Eq. 2), *recall* (Eq. 3), and *F1-Score* (Eq. 4). Accuracy is defined as the proportion of correct predictions relative to the total number of samples processed. Precision measures the system's ability to avoid false positives, while Recall evaluates its capacity to detect all instances of vulnerabilities. The F1-Score combines Precision and Recall into a single metric, giving more weight to the lower of the two values to provide a balanced measure of performance. To compute these metrics, we rely on the classification outcomes: *true positives* (TPs), which represent vulnerable samples correctly classified as such; *true negatives* (TNs), which refer to non-vulnerable samples correctly identified; *false positives* (FPs), denoting non-vulnerable samples misclassified as vulnerable; and *false negatives* (FNs), which are vulnerable samples incorrectly classified as non-vulnerable. These definitions are fundamental in deriving the equations for the performance metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

These metrics enable a comprehensive assessment of the model's performance, ensuring a balanced evaluation that accounts for both correctly identified and misclassified samples.

Table 2. Performance of DL approaches on MVFSC validation set.

Approach	Method	Accuracy	Precision	Recall	F1-Score
DL Methods	VulBERTa-MLP	0.83	0.91	0.83	0.86
	VulBERTa-TextCNN	**0.86**	**0.92**	**0.86**	**0.89**
	VulDetectionGNN	0.87	0.31	0.55	0.40

Table 3. Performance comparison of Traditional SAST tools, LLM methods, and DL methods on the MVFSC test set.

Approach	Method	Accuracy	Precision	Recall	F1-Score
Traditional SAST Tools	Semgrep	0.50	**0.54**	0.02	0.04
	Flawfinder	0.50	0.50	0.31	0.38
LLM Methods	CodeGeex4 9b	0.50	0.51	0.23	0.32
	QwenCoder 2.5 14b	0.50	0.52	0.22	0.30
	Code Llama 34b	**0.51**	0.51	**0.86**	**0.64**
DL Methods	VulBERTa-MLP	0.48	0.48	0.48	0.48
	VulBERTa-TextCNN	0.49	0.49	0.49	0.49
	VulDetectionGNN	0.49	0.50	0.57	0.53

The results presented in Table 2 and Table 3 offer important insights into the complexities of vulnerability detection. The consistently poor performance of traditional SAST tools on the test set underscores the inherent complexity of the task, as these tools fail to effectively identify vulnerabilities beyond basic

patterns, with precision and recall values only slightly surpassing random predictions. This highlights the limitations of rule-based approaches in handling the diverse and evolving nature of software vulnerabilities. DL methods, despite their strong performance on the validation set, demonstrate significant difficulty in generalizing to the test set. The drastic drop in metrics to near-random levels indicates an inability to adapt to unseen data distributions. The relatively better performance of VulDetectionGNN on the test set, reflected in its higher F1-Score, indicates that leveraging structural information, such as graph-based representations, may offer a more robust foundation for achieving generalization in previously unseen scenarios. The assessment of LLMs in a zero-shot setting, utilizing prompt engineering, highlights both challenges and opportunities for vulnerability detection. While the overall performance remains inadequate for practical deployment, increasing model complexity, as in the case of CodeLlama 34b, leads to a significant improvement in Recall and F1-Score. This suggests that larger models hold promise for enhancing detection capabilities, making this approach worth further exploration despite its current limitations. These findings align with the observations reported in [11], highlighting the intrinsic challenges of vulnerability detection in realistic scenarios. They emphasize the need for methodologies that effectively exploit structural information, such as graph-based representations, to improve model robustness and adaptability. Furthermore, the results reinforce the importance of advancing training and fine-tuning strategies to enhance the generalization capabilities of both deep learning models and LLMs.

5 Conclusions and Future Works

This study evaluated open-source LLMs for zero-shot vulnerability detection in C and C++ source code, highlighting their potential and limitations through prompt engineering techniques. Comparative analysis with SAST tools and deep learning methods revealed significant challenges in generalizing to real-world scenarios, emphasizing the need for robust benchmarks and improved training methodologies. Future research should prioritize high-quality datasets and explore hybrid approaches combining graph-based models, deep learning, and LLMs to enhance generalization, scalability, and reliability in critical domains like IoT security.

Acknowledgements. This work was supported in part by project SERICS (PE00000014) under the PNRR MUR program funded by the EU - NGEU.